Advanced Day 2 — Session 2 (Hours 5–8)

Python Programming (Advanced) • Factory Pattern, Strategy Pattern, Class Ergonomics, and Checkpoint 1

Session 2 Overview

Topics Covered Today

- Hour 5: Pattern — Factory (practical creation + validation)
- Hour 6: Pattern — Strategy (swap behaviors cleanly)
- Hour 7: Pythonic class ergonomics (`__repr__` / `__str__` / type hints)
- Hour 8: Checkpoint 1 — domain + service layer milestone

Source Alignment

Lecture Files Used in This Session

- `Advanced/lessons/lecture/Day2_Hour1_Advanced.md` → `# Day 2, Hour 1: Pattern: Factory`
- `Advanced/lessons/lecture/Day2_Hour2_Advanced.md` → `# Day 2, Hour 2: Pattern: Strategy`
- `Advanced/lessons/lecture/Day2_Hour3_Advanced.md` → `# Day 2, Hour 3: Pythonic Class Ergonomics`
- `Advanced/lessons/lecture/Day2_Hour4_Advanced.md` → `# Day 2, Hour 4: Checkpoint 1`

Runbook source of truth for all four hours:

`Advanced/Instructor/Python_Advanced_Instructor_Runbook_4hr_Days.md` → `## Session`

Scope Guardrails for Today

In Scope

- Centralized object creation and validation (Factory)
- Runtime behavior selection (Strategy)
- Readable `__repr__`, `__str__`, light type hints, `to_dict()`
- Checkpoint 1 — stable headless core: models + service CRUD + exceptions + serialization

Not Yet Covered

- Database persistence or file I/O (Day 3+)
- GUI callbacks or HTTP routes (later sessions)
- Advanced typing (`Protocol`, `TypeVar`, `Generic`)

Hour 5: Pattern — Factory

Learning Outcomes

- Explain why raw data should pass through one trusted creation path
- Implement `RecordFactory.from_dict()` for JSON-like dictionary input
- Enforce required fields, normalization, optional defaults, and value validation
- Raise `ValidationError` before an invalid object escapes the factory
- Keep the model constructor simple; put raw-input logic in the factory

Source Alignment

- `Advanced/lessons/lecture/Day2_Hour1_Advanced.md` → `## Learning Outcomes`

Why Factories Matter

Real programs receive raw, messy input:

```
{
    "title": "  write tests  ",
    "owner": "MARIA",
    "status": "IN_PROGRESS",
    "priority": "High"
}
```

Problems without a factory:

- Every feature writes its own parsing and validation rules
- Inconsistent defaults (`"done"` vs `"Done"` vs `"DONE"`)
- Bugs appear far from the cause when silent defaults hide missing data
- `__init__` becomes a noisy mix of storage, parsing, and error reporting

Factory Vocabulary

Term	Meaning
Factory	Code whose job is to create objects from raw input
Raw input	Data before our program trusts it
Normalization	Converting input into a consistent shape
Required field	Must be supplied; never silently defaulted
Optional field	May safely receive a default
Caller boundary	Where validation errors are caught and reported

Arrow: `raw dict → factory validates → valid object or ValidationError`

Source Alignment

- `Advanced/lessons/lecture/Day2_Hour1_Advanced.md` → `## Outcomes, Setup, and`

Demo: Simple Model + Factory Skeleton

```python
class ValidationError(Exception):
    """Raised when raw input cannot produce a valid record."""


class TaskRecord:
    def __init__(
        self,
        title: str,
        owner: str,
        status: str,
        priority: str,
        estimate_minutes: int,
    ) -> None:
        self.title = title
        self.owner = owner
        self.status = status
        self.priority = priority
        self.estimate_minutes = estimate_minutes

    def __str__(self) -> str:
        return (
            f"{self.title} | owner={self.owner} | status={self.status} | "
            f"priority={self.priority} | estimate={self.estimate_minutes}m"
        )
```

Demo: Full `RecordFactory.from_dict()`

```python
class RecordFactory:
    REQUIRED_FIELDS = ("title", "owner")
    ALLOWED_STATUSES = {"todo", "in_progress", "done"}
    ALLOWED_PRIORITIES = {"low", "normal", "high"}

    @classmethod
    def from_dict(cls, data: dict[str, object]) -> TaskRecord:
        for field_name in cls.REQUIRED_FIELDS:
            if field_name not in data:
                raise ValidationError(f"Missing required field: {field_name}")
        title = str(data["title"]).strip()
        owner = str(data["owner"]).strip().lower()
        if not title:
            raise ValidationError("Field 'title' cannot be blank")
        if not owner:
            raise ValidationError("Field 'owner' cannot be blank")
        status = str(data.get("status", "todo")).strip().lower()
        priority = str(data.get("priority", "normal")).strip().lower()
        if status not in cls.ALLOWED_STATUSES:
            raise ValidationError(f"Invalid status '{status}'")
        if priority not in cls.ALLOWED_PRIORITIES:
            raise ValidationError(f"Invalid priority '{priority}'")
        estimate_minutes = int(data.get("estimate_minutes", 0))
        return TaskRecord(title, owner, status, priority, estimate_minutes)
```

Demo: Running the Factory

```python
sample_records = [
    {"title": "  write factory tests  ", "owner": "MARIA",
     "status": "IN_PROGRESS", "priority": "High", "estimate_minutes": "45"},
    {"title": "Document validation rules", "owner": "jamal"},
    {"title": "Missing owner should fail", "status": "todo"},
]

for index, raw in enumerate(sample_records, start=1):
    try:
        record = RecordFactory.from_dict(raw)
        print(f"Record {index} loaded: {record}")
    except ValidationError as error:
        print(f"Record {index} rejected: {error}")
```

Expected output:

```
Record 1 loaded: write factory tests | owner=maria | status=in_progress | priority=high | estimate=45m
Record 2 loaded: Document validation rules | owner=jamal | status=todo | priority=normal | estimate=0m
Record 3 rejected: Missing required field: owner
```

Lab: Tracker Factory Implementation

Time: 25–35 minutes

Tasks

1. Define `ValidationError` and a simple model (`TrackerRecord` or `TaskRecord`)
2. Keep the constructor simple — store clean values only
3. Implement `RecordFactory.from_dict(data)`:
   - Check required fields (`title`, `owner`) — fail loudly if missing or blank
   - Normalize: strip whitespace, lowercase status/owner
   - Apply optional defaults: `status='todo'`, `priority='normal'`
   - Validate allowed values for status and priority
4. Load two valid hard-coded dictionaries through the factory
5. Show one invalid dictionary caught at the caller boundary

Common Pitfalls (Hour 5)

⚠️ Silently defaulting required fields — `title = data.get("title", "Untitled")` hides missing data

⚠️ All raw-dict parsing inside `__init__` — the constructor becomes a noisy traffic jam

⚠️ Factory raises a broad `Exception` instead of `ValidationError` — callers cannot distinguish errors

⚠️ Forgetting to validate after defaulting — a default status of `"todo"` is fine; accepting any status after defaulting is not

Source Alignment

- `Advanced/lessons/lecture/Day2_Hour1_Advanced.md` → `## Guided Lab` → `### Instructor Circulation Plan`

Quick Check (Hour 5)

Question: Why is `data.get("title", "Untitled")` dangerous for a required field, but `data.get("status", "todo")` safe for an optional field?

Hour 6: Pattern — Strategy

Learning Outcomes

- Define Strategy as selecting an algorithm at runtime
- Explain that Python strategies can be functions, lambdas, or callable objects
- Replace branchy `if` / `elif` blocks with a strategy registry dictionary
- Keep the core workflow stable while behavior changes
- Handle unknown choices explicitly rather than silently falling back

Source Alignment

- `Advanced/lessons/lecture/Day2_Hour2_Advanced.md` → `## Learning Outcomes`

Why Strategy Matters

Without Strategy, behavior accumulates in one place:

```
def display_records(records, sort_choice):
    if sort_choice == "name":
        ordered = sorted(records, key=lambda r: r["name"].lower())
    elif sort_choice == "created":
        ordered = sorted(records, key=lambda r: r["created_date"])
    elif sort_choice == "priority":
        ordered = sorted(records, key=lambda r: r["priority"])
    else:
        raise ValueError(f"Unknown sort choice: {sort_choice}")
    for record in ordered:
        print(record)
```

- Every new sort rule requires editing this function
- The same branching often appears in multiple display surfaces
- One place strips case; another forgets — bugs appear

Strategy Vocabulary

Term	Meaning
Strategy	A replaceable algorithm — a callable with the expected shape
Runtime selection	Choosing the strategy while the program runs
Callable	Any Python object you can call with `()`
Core workflow	The stable part of the code that receives and uses a strategy
Strategy registry	A dictionary mapping user choices to strategy callables

Arrow: `choice → registry lookup → core workflow receives strategy → result changes`

Source Alignment

- `Advanced/lessons/lecture/Day2_Hour2_Advanced.md` → `## Outcomes, Setup, and`

Demo: Sort Key Strategies

```python
from typing import Callable

Record = dict[str, object]
SortKey = Callable[[Record], object]

def sort_by_name(record: Record) -> str:
    return str(record["name"]).lower()

def sort_by_created_date(record: Record) -> str:
    return str(record["created_date"])

def sort_by_priority(record: Record) -> int:
    return int(record.get("priority", 999))

SORT_STRATEGIES: dict[str, SortKey] = {
    "1": sort_by_name,
    "2": sort_by_created_date,
    "3": sort_by_priority,
}

def choose_sort_strategy(choice: str) -> SortKey:
    try:
        return SORT_STRATEGIES[choice]
    except KeyError as error:
        raise ValueError(f"Unknown sort choice {choice!r}") from error

def display_records(records: list[Record], sort_key: SortKey, label: str) -> None:
    print(f"\nSorted by {label}:")
    for record in sorted(records, key=sort_key):
        print(f"- {record['name']} | {record['created_date']} | priority {record.get('priority', '-')}")
```

Demo: Validation Strategies

```python
Validator = Callable[[Record], list[str]]

def validate_task(record: Record) -> list[str]:
    errors: list[str] = []
    if not str(record.get("title", "")).strip():
        errors.append("task requires a non-empty title")
    if record.get("status") not in {"pending", "doing", "done"}:
        errors.append("task status must be pending, doing, or done")
    return errors

def validate_contact(record: Record) -> list[str]:
    errors: list[str] = []
    if not str(record.get("name", "")).strip():
        errors.append("contact requires a non-empty name")
    if "@" not in str(record.get("email", "")):
        errors.append("contact requires an email containing @")
    return errors

VALIDATION_STRATEGIES: dict[str, Validator] = {
    "task": validate_task,
    "contact": validate_contact,
}

def report_validation(record: Record, validator: Validator) -> None:
    errors = validator(record)
    label = record.get("name", "<unnamed>")
    if errors:
        print(f"{label} failed: {errors}")
    else:
        print(f"{label} passed.")
```

Demo: Callable Class Strategy with State

```python
class NamePrefixFilter:
    def __init__(self, prefix: str) -> None:
        self.prefix = prefix.lower()

    def __call__(self, record: Record) -> bool:
        return str(record["name"]).lower().startswith(self.prefix)


def include_all(record: Record) -> bool:
    return True


def filter_records(
    records: list[Record],
    keep: Callable[[Record], bool],
) -> list[Record]:
    return [record for record in records if keep(record)]


starts_with_a = NamePrefixFilter("A")
print("All records:", filter_records(records, include_all))
print("Names starting with A:", filter_records(records, starts_with_a))
```

Lab: Menu-Selected Strategies

Time: 25–35 minutes

Tasks

1. Choose a behavior that varies: sorting by name vs created date, or filtering active vs all
2. Implement at least **two** strategies as separate functions or callable objects
3. Store strategies in a dictionary registry
4. Wire a simulated menu choice to select the strategy
5. Pass the selected strategy into a stable core function
6. Run with two different choices — output must be clearly different
7. Handle an unknown choice explicitly (raise `ValueError`, not silent fallback)

Fast finisher: combine a sort strategy and a filter strategy; add a third strategy

Common Pitfalls (Hour 6)

⚠️ Hard-coded branching still lives in the core workflow — strategy names exist but selection moved, not eliminated

⚠️ Accidentally calling the function in the registry — `"1": sort_by_name()` stores the result, not the callable

⚠️ Strategies that read hidden global variables — strategy state must come from parameters or constructor args

⚠️ Silent fallback on unknown choice — `SORT_STRATEGIES.get(choice, sort_by_name)` hides typos and bugs

Source Alignment

- `Advanced/lessons/lecture/Day2_Hour2_Advanced.md` → `## Troubleshooting and Common Pitfalls`

Quick Check (Hour 6)

Question: How is using a strategy dictionary different from an `if` / `elif` chain, and why does it matter when a new sort option is added?

Hour 7: Pythonic Class Ergonomics

Learning Outcomes

- Explain why the default `<__main__.Task object at 0x...>` output is not enough
- Implement `__repr__` for developer-facing debug inspection
- Implement `__str__` for user-facing friendly display
- Add light type hints using simple built-in annotations
- Add `to_dict()` as checkpoint-prep serialization readiness

Source Alignment

- `Advanced/lessons/lecture/Day2_Hour3_Advanced.md` → `## Learning Outcomes`

Why Class Ergonomics Matter

Without ergonomics:

```
[<__main__.Task object at 0x000001F2A8C4D910>,
 <__main__.Task object at 0x000001F2A8C4DA50>]
```

With ergonomics:

```
[Task(task_id=101, title='Write tests', status='open', priority=2),
 Task(task_id=102, title='Review PR', status='blocked', priority=1)]
```

- Debugging a wrong sort order is now possible at a glance
- Logs include meaningful state, not memory addresses
- Checkpoint reviewers can verify output without opening the debugger

Source Alignment

`__repr__` vs `__str__`

Method	Audience	When used
`__repr__`	Developer	`repr(obj)`, list/dict containers, REPL, debugger
`__str__`	User	`print(obj)`, `str(obj)`, CLI menus, reports

Rule of thumb:

- `__repr__` should be specific and unambiguous — include all important fields
- `__str__` should be short and friendly — only what the user needs
- If `__str__` is missing, Python falls back to `__repr__` for `print()`

```
repr(task)  # Task(task_id=101, title='Write tests', status='open', priority=2)
str(task)   # #101 Write tests [open]
print([task])  # uses __repr__ — lists always use repr for contents
```

Demo: Adding `__repr__` and `__str__`

```
class Task:
    def __init__(self, task_id: int, title: str, status: str, priority: int) -> None:
        self.task_id = task_id
        self.title = title
        self.status = status
        self.priority = priority

    def __repr__(self) -> str:
        return (
            "Task("
            f"task_id={self.task_id!r}, "
            f"title={self.title!r}, "
            f"status={self.status!r}, "
            f"priority={self.priority!r}"
            ")"
        )

    def __str__(self) -> str:
        return f"#{self.task_id} {self.title} [{self.status}, priority {self.priority}]"
```

Note `!r` inside the f-string — it calls `repr()` on the value, adding quotes to strings

Demo: Type Hints + `to_dict()` for Checkpoint Readiness

```
    def is_complete(self) -> bool:
        return self.status == "done"

    def rename(self, new_title: str) -> None:
        self.title = new_title.strip()

    def to_dict(self) -> dict[str, object]:
        return {
            "task_id": self.task_id,
            "title": self.title,
            "status": self.status,
            "priority": self.priority,
        }


def format_task_for_log(task: Task) -> str:
    return f"Saving {task!r}"
```

Expected output snippet:

Lab: Make Your Model Pleasant to Work With

Time: 25–35 minutes

Tasks (in order)

1. Choose your main model (`Task` , `Contact` , `InventoryItem` , or your tracker class)
2. Add or improve `__repr__` — include debug-important fields, use `!r` for string values
3. Add or improve `__str__` — short, user-readable, suitable for CLI or report output
4. Add type hints to at least three methods (include the constructor)
5. Add or refine `to_dict()` — required for the Checkpoint 1 lab next hour

Output check:

- `print(obj)` → user-friendly
- `repr(obj)` → developer-friendly with field names

Completion Criteria (Hour 7)

✓ `__repr__` returns a string with class name and key fields (uses `!r` for values)
✓ `__str__` returns a short, user-friendly string — does not call `print()` internally
✓ At least three type hints are present and correct
✓ `to_dict()` returns a dictionary with stable, meaningful keys
✓ `print(obj)` and `repr(obj)` produce clearly different, useful output

Common Pitfalls (Hour 7)

⚠️ `__str__` calls `print()` instead of `return` — returns `None`, raises `TypeError` at runtime

⚠️ Over-complicated typing imports — `from typing import Any, Callable, Optional, TypeVar...` is not needed for this hour; use simple built-ins

⚠️ `to_dict()` omits important fields or returns inconsistent types — checkpoint reviewers need stable keys

⚠️ Treating hints as runtime validation — `priority: int` does not prevent passing `"high"` unless explicit validation exists

Source Alignment

- `Advanced/lessons/lecture/Day2_Hour3_Advanced.md` → `## Troubleshooting and Common Pitfalls`

Quick Check (Hour 7)

Question: A list of tasks prints with `repr` output for each task, even though you defined `__str__`. Why does that happen, and when would you explicitly call `str(task)`?

Hour 8: Checkpoint 1 — Domain + Service Layer Milestone

Learning Outcomes

- Explain what "core tracker library" means: domain + service, no UI or database inside
- Identify the minimum Checkpoint 1 requirements
- Demonstrate a valid happy path and at least two sad paths
- Keep domain and service responsibilities separate from UI responsibilities
- Reflect on one design decision and explain why it helps the code

Source Alignment

- `Advanced/lessons/lecture/Day2_Hour4_Advanced.md` → `## Learning Outcomes`

What "Core Tracker Library" Means

```
Domain model:   What thing does the app manage?
Service layer:  What operations can the app perform on those things?
UI/API/DB:      Later layers — they attach TO the core, not live INSIDE it.
```

- Service methods accept arguments, return values, raise exceptions
- Service methods do **not** call `input()` or `print()` internally
- A GUI button could call `service.add(task)` later — without rewriting the model
- An API endpoint could call `service.update(id, **changes)` later — without changes

Source Alignment

- `Advanced/lessons/lecture/Day2_Hour4_Advanced.md` → `## Opening Bridge and Checkpoint Framing`

Checkpoint 1 Rubric

Category	Yes evidence
Models/helpers	2+ model classes, or 1 model + 1 helper; constructors validate key fields
Service CRUD	add, list, search, update, delete — callable from a demo script
Custom exceptions	`ValidationError` + `NotFoundError`; expected errors caught in demo
Serialization	`to_dict()` on the main model; enough fields for save or API response
Runnable demo	Runs end-to-end; proves happy path + at least 2 sad paths
Separation of concerns	No `input()` inside service methods; printing stays in demo/wrapper

Demo: Checkpoint Fast Grading Demo

```python
from dataclasses import dataclass

class ValidationError(Exception):
    pass

class NotFoundError(Exception):
    pass

@dataclass
class Task:
    task_id: int
    title: str
    owner: str
    status: str = "todo"
    priority: int = 3
    estimate_minutes: int = 30
    VALID_STATUSES = {"todo", "doing", "done", "blocked"}

    def __post_init__(self) -> None:
        if not self.title.strip():
            raise ValidationError("title is required")
        if self.status not in self.VALID_STATUSES:
            raise ValidationError(f"status must be one of {sorted(self.VALID_STATUSES)}")

    def to_dict(self) -> dict[str, object]:
        return {"task_id": self.task_id, "title": self.title,
                "owner": self.owner, "status": self.status,
                "priority": self.priority, "estimate_minutes": self.estimate_minutes}
```

Demo: TaskFactory + TaskService

```python
class TaskFactory:
    @staticmethod
    def from_dict(data: dict[str, object]) -> Task:
        try:
            return Task(task_id=int(data["task_id"]), title=str(data["title"]),
                        owner=str(data["owner"]),
                        status=str(data.get("status", "todo")),
                        priority=int(data.get("priority", 3)),
                        estimate_minutes=int(data.get("estimate_minutes", 30)))
        except KeyError as exc:
            raise ValidationError(f"missing required field: {exc.args[0]}") from exc

class TaskService:
    def __init__(self) -> None:
        self._tasks: dict[int, Task] = {}
    def add(self, task: Task) -> Task:
        self._tasks[task.task_id] = task
        return task
    def list_all(self) -> list[Task]:
        return list(self._tasks.values())
    def search(self, text: str) -> list[Task]:
        text = text.casefold()
        return [t for t in self._tasks.values()
                if text in t.title.casefold() or text in t.owner.casefold()]
    def get(self, task_id: int) -> Task:
        try:
            return self._tasks[task_id]
        except KeyError as exc:
            raise NotFoundError(f"task id {task_id} was not found") from exc
    def update(self, task_id: int, **changes: object) -> Task:
        updated_data = self.get(task_id).to_dict()
        updated_data.update(changes)
        updated = TaskFactory.from_dict(updated_data)
        self._tasks[task_id] = updated
        return updated
    def delete(self, task_id: int) -> Task:
        task = self.get(task_id)
        del self._tasks[task_id]
        return task
```

Lab: Build and Grade Checkpoint 1

Time: 40 minutes

Minimum Requirements

1. **Models/helpers** — 2+ model classes, or 1 model + 1 helper class (e.g., `TaskFactory`)
2. **Service CRUD** — add, list, search, update, delete; no `input()` inside service
3. **Custom exceptions** — `ValidationError` for bad data, `NotFoundError` for missing records
4. **Serialization** — `to_dict()` returns a dictionary with meaningful fields
5. **Runnable demo** — proves happy path + invalid input + missing record

Demo shape:

```
print("--- Checkpoint 1 Demo ---")
```

Completion Criteria (Hour 8)

✓ Core runs end-to-end through a small demo script
✓ Model/helper structure is present and responsible
✓ Service can add, list, search, update, and delete without requiring UI input
✓ Invalid input raises `ValidationError`; missing records raise `NotFoundError`
✓ `to_dict()` returns a stable dictionary
✓ Service logic is not trapped inside UI prompts or global script flow

Common Pitfalls (Hour 8)

⚠️ Global state — `global tasks` in service functions makes each call share state unexpectedly; use a service object with `self._tasks`

⚠️ `input()` inside service methods — cannot be graded quickly, cannot be reused by GUI or API

⚠️ Update without re-validation — changing `status` through direct attribute assignment can bypass invariants; rebuild through the factory

⚠️ Demo proves only the happy path — missing a `ValidationError` or `NotFoundError` catch means the checkpoint is incomplete

Source Alignment

- `Advanced/lessons/lecture/Day2_Hour4_Advanced.md` → `## Troubleshooting and Pitfalls`

Quick Check (Hour 8)

Question: Could someone attach a GUI button to your `service.update(task_id, status="done")`
without rewriting the model? What would prevent that?

Session 2 Wrap-Up

Key Takeaways from All Four Hours

Hour 5 — Factory

- One trusted creation path from raw data to valid objects
- Required fields fail loudly; optional fields default intentionally

Hour 6 — Strategy

- Extract variable behavior into named callables; keep the core workflow stable
- A strategy registry dictionary replaces branchy dispatch

Hour 7 — Ergonomics

- `__repr__` for developers; `__str__` for users; both must `return` a string
- Light type hints document intent; `to_dict()` prepares for serialization

Source Alignment — All Files Used

Lecture File	Content Covered
`Advanced/lessons/lecture/Day2_Hour1_Advanced.md`	Factory pattern, `RecordFactory.from_dict()`, required fields, normalization, defaults, validation, lab, pitfalls
`Advanced/lessons/lecture/Day2_Hour2_Advanced.md`	Strategy pattern, sort-key strategies, validation strategies, `NamePrefixFilter` callable class, strategy registry, lab, pitfalls
`Advanced/lessons/lecture/Day2_Hour3_Advanced.md`	`__repr__`, `__str__`, type hints, `to_dict()`, equality sidebar

Scope Guardrail — What Is NOT Covered Here

Topics Deferred to Later Sessions

- **Database persistence** — JSON save/load, SQLite, or ORM (Day 3+)
- **Package structure** — `src/` layout, `__init__.py`, import discipline (Day 3)
- **Logging and diagnostics** — `logging` module, structured log output (Day 3)
- **Context managers** — `with` statements for safe file/resource handling (Day 3)
- **Decorators** — timing, auth, memoization (Day 3)
- **GUI or HTTP layers** — attach to the stable core in later sessions
- **Advanced typing** — `Protocol`, `TypeVar`, `Generic`, `mypy` configuration

Not Out of Scope Forever

These topics build directly on the core established in this session.

Next Session Preview

Session 3 (Hours 9–12) — Day 3

- **Hour 9** — Package structure under `src/`: modules, `__init__.py`, import discipline
- **Hour 10** — Logging and developer diagnostics: `logging` module, structured output
- **Hour 11** — Context managers and safe file patterns: `with` statements, custom managers
- **Hour 12** — Decorators: timing, validation wrappers, practical patterns

Your Checkpoint 1 core becomes the package that Session 3 structures, logs, and wraps.

Questions?

Save your work.

Commit the Checkpoint 1 state.

Come back ready to package the core properly.